

Node.js — Developing a minimally HashDoS resistant, yet quickly reversible integer hash for V8

Node.js — Developing a minimally HashDoS resistant, yet quickly reversible integer hash for V8 - @nodejs, nodejs.org.

- Published: date not stated
- Original: <<https://nodejs.org/en/blog/vulnerability/march-2026-hashdos>>
- Preserved from: <https://nodejs.org/en/blog/vulnerability/march-2026-hashdos> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Commercial support for versions past the Maintenance LTS phase is available through our [OpenJS Ecosystem Sustainability Program partners](#)

Developing a minimally HashDoS resistant, yet quickly reversible integer hash for V8

JC

Joyee Cheung

What happens when a hashing scheme needs to be both HashDoS resistant and quickly reversible? That's the puzzle we tried to solve for addressing [CVE-2026-21717](#) in the [March 2026 Node.js security release](#). This led to the development of an integer hash that we believe is unpredictable enough to prevent a blind attacker from reliably triggering severe performance degradation in our threat model. At the same time, it is also a [permutation](#) that can be efficiently inverted to recover the original integer value by the runtime holding the secret random keys, which is important for maintaining V8's performance optimizations.

In this post, we will go into the details of this vulnerability, explain why and how the design tried to meet the two seemingly contradictory requirements, and sample the statistical analysis we performed to evaluate the quality of the hash. Along the way, we will also discuss some specifics of how V8 stores and uses string hashes internally that lead to these constraints, the implementation techniques, and the performance analysis.

What is HashDoS and why does it matter for Node.js?

[Hash tables](#) are one of the most important data structures in software, and Node.js/V8 are no exceptions. With hash tables, there comes the problem of [collisions](#) - while hash tables are designed to have $O(1)$ average time complexity for, e.g., single lookup and insertion, when there are so many keys whose hashed values can be mapped to the same slot that the table has to walk through many locations to insert a new key, it's theoretically possible to degrade to $O(n^2)$ in total for inserting n keys.

In many applications, the worst-case performance of hash tables is usually just a theoretical concern and maybe an occasional nuisance, but for servers that need to process untrusted input, this can be an attack vector. If an attacker manages to find a path that allows them to cause a large number of collisions in the server's internal hash table with a small number of inputs, they can effectively "freeze" the thread without much effort. And, with enough threads frozen, they could render the service unavailable for a sustained period of time. This is commonly called a [hash flooding or Hash-DoS \(Denial of Service\) attack](#).

HashDoS vulnerabilities can show up in a wide range of platforms. In the case of V8, when it's used in a browser, DoS issues are [not considered vulnerabilities](#) because they typically only affect the tab that processes the malicious input, and users can simply close the tab. But when it's used in event-loop-based server-side runtimes, a single malicious request that manages to block the entire event loop for a sustained period of time has a much bigger impact. As one of the most widely deployed server-side runtimes, Node.js draws extensive scrutiny from security researchers, which helps us find and address weak spots like these.

Unlike other DoS attack vectors that can be more easily mitigated by userland restrictions, HashDoS vulnerabilities are particularly tricky because they lurk inside widely used internal data structures and well-defined operations trusted by developers. For V8 embedders, this means fixes for HashDoS vulnerabilities usually fall on the runtime maintainers, and they are a critical part of maintaining the security and reliability of the ecosystem.

Mitigating HashDoS with seeded hashes

The core of HashDoS vulnerabilities lies in deterministic hash functions - if the attacker can predict the hash values of their inputs, they can craft inputs that reliably trigger worst-case performance in the target server. The standard mitigation is to [mix a seed value into the computation of the hash, and make sure that the seed is randomly generated at program initialization](#). This way, even if the attacker knows the hash function, they can't predict the hash values without knowing the seed that changes whenever the server restarts, and therefore can't craft inputs that will reliably collide.

In Node.js/V8, HashDoS vulnerabilities tend to center around misconfigurations that lead to unseeded hashes or hashes seeded by a constant value. For an early history of HashDoS vulnerabilities in Node.js/V8 and their mitigations, check out this [V8 blog post](#). A more recent example is [CVE-2025-27209](#), reported by Mate Marjanović and addressed in Node.js's July 2025 security release: during V8's switch to [rapidhash](#), a configuration gap left the constants used by rapidhash hard-coded in V8 without seeding. To mitigate this, Node.js temporarily reverted back to a seeded hash in its copy of V8. Later, Gus Caplan from Deno [ported rapidhash's secret generation](#)

code to V8 and wired it up to seed the constants used by rapidhash at runtime, which became the eventual mitigation that shipped in newer versions of Node.js. The generation of rapidhash secrets turned out to be important to our later design for the seeded integer hash, which we will discuss below.

Until recently, most HashDoS discoveries in V8 had centered on hashing of regular strings, since they are more commonly found in paths exposed to server handlers. It wasn't until the recent HackerOne report by Mate Marjanović that our attention was drawn to an elephant in the room: array index strings (those that look like non-negative integers and fit within 24 bits) in V8 used a different deterministic hash that can be trivially predicted.

What string hashes look like in V8

To understand the vulnerability, let's look at how V8 stores string hashes internally.

Every `v8::internal::Name` object (the superclass of `v8::internal::String` in V8) carries a 32-bit `raw_hash_field`. The composition of this field varies depending on the type of string. The bottom 2 bits (`HashFieldIdTypeBits`) determine the interpretation, and two of the most common types are:

- Regular strings (type `kHash = 0b10`): bits 2-31 store a 30-bit hash value computed by rapidhash, seeded with randomly generated secrets. This is considered to be "minimally HashDoS resistant", which we will discuss later.
- Array index strings (type `kIntegerIndex = 0b00`): these are decimal-integer-looking strings whose numeric value fits in 24 bits. bits 2-25 (24 bits) hold the numeric value of the integer (`ArrayIndexValueBits`), and bits 26-31 (6 bits) hold the string length (`ArrayIndexLengthBits`).

The layout of the hash fields looks like this (the least significant bits are on the right):

```
"hello" (type: kHash = 0b10):
+-----+-----+
| 30-bit rapidhash | 1 0 |
+-----+-----+
31           2 1 0

"1234" (length=4, value=1234, type: kIntegerIndex = 0b00):
+-----+-----+
| 000100 | 0000000000000010011010010 | 0 0 |
| length | numeric value | |
+-----+-----+
31  26 25           2 1 0
```

As you can see, the hash for array index strings is fully deterministic and there's no seeding involved. An attacker can easily predict their hash values, and with a bit of effort, craft colliding inputs.

Exploiting the deterministic hash for array index strings

One of the most commonly used hash tables in V8 is the [string table](#), which is used for [string inter nalization](#) - deduplicating and caching strings in a global table for time and memory efficiency. The string table uses [open addressing](#) with [quadratic probing](#) to deal with collisions. The [probing sequence](#) is:

$$\text{slot0} = \text{hash} \& (\text{capacity} - 1), \text{capacity} = 2^n, \text{slotk} = (\text{slotk} - 1 + k) \& (\text{capacity} - 1) = (\text{slotk} - 1 + k) \pmod{\text{capacity}}$$

The string table [grows by adding 50% slack and rounding the capacity to a power of two](#), so $\text{hash} \& (\text{capacity} - 1)$ equals $\text{hash} \pmod{\text{capacity}}$, and two strings with congruent hashes land on the same first-probe slot. For array index strings, the hash is $(\text{length} \ll 24) \mid \text{value}$ - but because the capacity is usually smaller than 2^{24} , the length bits are usually masked away and the slot depends only on the numeric value mod capacity. Take a table with initial capacity 32768 for example:

String	Hash	Hash & (capacity - 1)	Slot
"1234"	$(4 \ll 24) \mid 1234$	$0x040004D2 \& 32767$	1234
"34002"	$(5 \ll 24) \mid 34002$	$0x050084D2 \& 32767$	1234
"525522"	$(6 \ll 24) \mid 525522$	$0x060804D2 \& 32767$	1234

All three collide on slot 1234. The attacker can fill the probing sequence from the target value's first-probe slot to cause severe performance degradation, as demonstrated in the PoC in the [HackerOne report](#).

```
const payload = [];  
const val = 1234;  
const MOD = 2 ** 19;  
const CHN = 2 ** 17; // chain length  
const REP = 2 ** 17; // repetitions of the target value  
  
// Build the quadratic probing chain  
let j = val + MOD;  
for (let i = 1; i < CHN; i++) {  
  payload.push(`${j}`);  
  j = (j + i) % MOD;  
}  
  
// Repeat the target value to force lookups through the chain  
for (let k = 0; k < REP; k++) {  
  payload.push(`${val}`);  
}  
  
// On the client side: attacker crafts an adversarial JSON payload  
// and sends it to the remote server.  
const string = JSON.stringify({ data: payload });  
  
// On the server side: V8 inserts the numeric strings in a hash table  
// for internalization, the collisions lead to extreme amplification  
// in resource consumption.  
JSON.parse(string);
```

With a ~2 MB payload, this can significantly slow down the JSON parsing - even on a powerful MacBook, this can hang for about half a minute. This means for servers that parse JSON from untrusted sources, a remote attacker can cause significant disruption via extreme [asymmetric resource consumption](#). Since the same hashing scheme is also used for many other V8 internals, e.g., Map keys, this has a wide attack surface.

HashDoS resistant vs. efficiently reversible

As mentioned above, the standard mitigation for HashDoS is to seed the hash function, so we need to look for a more robust, seeded hashing scheme for array index strings.

When we talk about hashes for security purposes, we often naturally think of [cryptographic hashes](#) - which are, by design, irreversible. And here we have a dilemma: V8's array index hash is not just a hash - it's a reversible encoding. This enables an important optimization that happens everywhere in V8: for example, in many fast paths that involve string-to-integer conversion, like `parseInt("42")` or `obj["42"] = 1`, instead of trying to parse the number from the string (whose content is not necessarily in CPU cache), V8 simply reads the `raw_hash_field` of the string and extracts the numeric value directly from the hash field. V8 also takes advantage of this encoding in e.g., string equality checks, where it would just compare two integer strings by their hashes. By nature, an irreversible cryptographic hash would break these optimizations and could lead to significant performance regressions in many hot paths.

So it looks like we are in a bind: do we have to either live with this vulnerability because of the reversibility requirement, or accept the performance regression?

Finding a middle ground: minimal HashDoS resistance

Now is the time to take a step back and think about what exactly makes a hash resistant to HashDoS attacks. It turned out that those who looked into rapidhash wondered the same and proposed the idea of "[minimal HashDoS resistance](#)", with constraints that apply to our case:

- The attacker cannot observe the hash output: The hash values are internal to V8 and not exposed directly to JavaScript or network responses. The attacker has to work blind.
- [Node.js trusts the code it is asked to run and the infrastructure that runs it](#). The attacker should be sending untrusted input from a remote machine, and network jitter, garbage collection latency, server load etc. typically render the cost of exploiting timing side channels prohibitive.

In the specific context of Node.js/V8 servers, there are also a few other important factors in play:

- The attacker needs to work with a noisy table with application-specific entries out of their control. It is shared by both array-index strings and regular strings hashed by rapidhash, and the capacity growth can also be affected by pre-existing entries, making it harder for the attacker to predict the exact probe chain consistently.
- Standard operational practices in production servers typically enforce payload size limits and rate limits. Without them, the server can already be exposed to simpler resource exhaustion attacks. These bound how many collisions an attacker can inject per request and how often they can retry. Other common practices like load balancing can further complicate the attacker's ability to accumulate collisions on the same server.

- Hash table entries held weakly are subject to V8's garbage collection. For example, between requests, internalized strings that become unreachable are cleaned, which helps prevent collisions from accumulating across requests and limit what an attacker can achieve in each payload. If the server allows an attacker to keep unlimited entries alive across GC cycles, no hash is strong enough to fix the off-table memory growth and it's that unbounded retention path that needs to be fixed.

These contributed to the evaluation of high attack complexity in [CVE-2026-21717](#) since the extreme amplification cannot be consistently reproduced against all Node.js servers, e.g., an Express server with its default [100 KB body limit](#).

That said, for servers with fewer guardrails or simpler states, a fully deterministic hash still makes the attack feasible. So while we don't need the hash to be cryptographically secure (or due to the reversibility requirement it just can't be), we still need to find an efficiently invertible, randomly keyed permutation with good diffusion on the 24-bit space, in order to bring the unpredictability of array index string hashes up to a level similar to randomly keyed rapidhash's unpredictability for regular strings.

Exploring candidate hashes

Some naive hashes and why they fail

We first prototyped with a few naive hashes, partly to identify all the code paths in V8 that would need updating, and partly to see whether something simple might be good enough in practice.

Multiply a secret, then add another

```
const uint32_t kMask = (1 << 24) - 1; // 24-bit mask
uint32_t SeedArrayIndexValue(uint32_t value, uint32_t secrets[2]) {
    return (secrets[0] * value + secrets[1]) & kMask;
}
```

The first naive idea that came to mind was the classic [linear congruential generator](#) (LCG) construction, but applied individually on the input instead of on a sequence. This is bijective when `secrets[0]` is odd, and can be quickly inverted (just subtract and multiply by the [modular inverse](#)). Unfortunately, this construction preserves linear relationships, and the low bits of the output depend only on the low bits of the input. As mentioned before, in many hash tables only the lower bits matter in probing, so an attacker can still generate collisions by picking values that are congruent modulo a guessed capacity.

XOR with a secret

```
uint32_t SeedArrayIndexValue(uint32_t value, uint32_t secrets[1]) {
    return value ^ secrets[0]; // Both secrets and values are already 24-bit.
}
```

Another suggestion that came up during the development to minimize the overhead was to XOR the value with the randomly generated seed. This is also bijective and the inversion is extremely fast (just XOR the secret again), but each bit of the output depends on exactly one bit of the input. An attacker can still construct collisions just by choosing values that agree in the low bits.

These were clearly not good enough. We needed something with genuine bit diffusion, where changing a single input bit would affect many output bits in an unpredictable way for the attacker.

The xorshift-multiply mixers

The search for bijective integer hash functions led us to Christopher Wellons' [hash-prospector](#) project, which generates billions of integer hash functions at random from a selection of nine reversible operations, then evaluates and ranks them. This project [showed](#) that many of the best-performing functions came from the same family of constructions that use alternating rounds of two operations:

- $x_{i+1} = x_i \gg k$ (right [xorshift](#)): this is bijective and invertible. The top k bits are unchanged, and we can recover the next k bits by XORing with them, then repeat until all bits are restored. A particularly elegant aspect of xorshift is that when the shift k is at least half the bit width, this is an [involution](#): $x_{i+1} = x_i \gg k$. It is linear over [GF\(2\)](#), but nonlinear over [/2N](#) .
- $y_{i+1} = y_i \times m \pmod{2N}$ (multiplication): this is bijective when m is odd, and invertible by multiplying with minv , the modular inverse of $m \pmod{2N}$. It is linear over [/2N](#) , but nonlinear over [GF\(2\)](#).

This family of xorshift-multiply constructions is used in many pseudorandom number generators/mixers like Java's [SplittableRandom](#) and [MurmurHash3's finalizer](#). A great explanation of this construction can be found in [this article](#) (and [this](#)): multiplication propagates information upward through its carry chain, while XOR-shift propagates information downward by folding high bits into low positions. By [mixing operations from different algebraic groups](#), this construction helps break the patterns each one preserves.

Since our multipliers are random secrets, the interaction between the two operations differs for each set of secrets, and multiple rounds of alternation helps spread the uncertainty across all bits. Applying this structure to our 24-bit input space, we first came up with the following design:

```
const uint32_t kMask = (1 << 24) - 1; // 24-bit mask
const uint32_t kShift = 12; // half the bit width, so the xorshift is an involution
// Multipliers in m[] are odd and randomly generated at startup.
uint32_t SeedArrayIndexValue(uint32_t value, uint32_t m[2]) {
    uint32_t m1 = m[0], m2 = m[1];
    uint32_t x = value;
    x ^= x >> kShift; x = (x * m1) & kMask; // round 1
    x ^= x >> kShift; x = (x * m2) & kMask; // round 2
    x ^= x >> kShift; // finalize
    return x;
}
```


To recover the original value, we can simply apply the steps in reverse order, replacing each multiplier with its modular inverse. This looks nicely symmetric:

```
// Modular inverses in m_inv[] are precomputed using Newton's method at startup.
uint32_t UnseedArrayIndexValue(uint32_t hash, uint32_t m_inv[2]) {
    uint32_t m1_inv = m_inv[0], m2_inv = m_inv[1];
    uint32_t x = hash;
    x ^= x >> kShift; x = (x * m2_inv) & kMask; // undo round 2
    x ^= x >> kShift; x = (x * m1_inv) & kMask; // undo round 1
    x ^= x >> kShift; // finalize
    return x;
}
```

Multiplier generation

Now that we had found a(nother) plausible structure to mix the bits, the next step was to find a way to generate good multipliers. The multipliers need to be 24-bit since the permutation needs to operate on the 24-bit modular arithmetic space, and they also need to be chosen well - if the multiplier is 1, for example, it won't mix the bits at all. On the other hand, since they need to be generated at startup, we need to minimize the cost of finding good multipliers.

We turned to the existing rapidhash secrets that V8 already generates at startup, which appeared to be a decent fit for our needs. While there is a width mismatch (rapidhash secrets are 64-bit), the secrets still retain some [desirable properties](#) after truncation. For example, each byte in the secrets must have exactly 4 bits set, and the generation ensures the secrets are odd, which is needed for the permutation to work. So we ended up just taking the lowest 24 bits of each rapidhash secret to derive the multipliers:

```
const uint32_t kMask = (1 << 24) - 1; // 24-bit mask
uint32_t derive_multiplier(uint64_t secret) {
    // The | 1 ensures the multiplier is odd, which is redundant in practice
    // but serves as a safeguard.
    return ((uint32_t)secret & kMask) | 1;
}
```

And because V8 already has to generate them anyway, we essentially get these multipliers for free and can reuse a good chunk of the infrastructure around its management.

Statistical evaluation

With the construction taking shape, we'd like to see some empirical evidence about how well it diffuses the bits. While our threat model primarily relies on the secrecy of the multipliers and the invisibility of hash output, good diffusion is needed for that secrecy to reach every output bit. One common way to quantify diffusion for a hash function is to check its [avalanche effect](#), which

measures how a small change in the input affects the output bits. For example, if for each input x and each input bit position j , we compute the hash of both x and x with bit j flipped, then count how often each output bit k changed, the ideal hash function should have each output bit flipped 50% of the time for each input bit flip, known as the [strict avalanche criterion \(SAC\)](#). We adapted the code in [hash-prospector](#) to evaluate the bias (root-mean-square relative deviation) from the SAC for our 24-bit input space (scaled by 1000 for readability):

$$\text{SAC} = 2N - 1 \text{bias} = 1000 \cdot \frac{1}{N^2} \sum_{j,k} (c_{j,k} - \text{SAC})^2$$

where $N=24$ is the bit width. A bias of 0 means perfect avalanche, and 1000 means zero diffusion.

We ran this on the original deterministic hash (identity function) as a baseline, then on a few naive ideas mentioned above, and finally on 1 and 2 rounds of xorshift-multiply with rapidhash's default secrets as multipliers:

Construction	Bias
identity	1000.000
xor only	1000.000
mul+add	797.523
1-round xorshift-multiply	446.852
2-round xorshift-multiply	3.447

While there's still room for improvement, it already looked very promising, so this was what our [initial implementation](#) used. But since our multipliers are derived from randomly generated rapidhash secrets, how do we know that the avalanche properties will be consistently good across different randomly generated secrets? We needed to verify that as well.

We [ran the same analysis on multipliers derived from 50 sets of randomly generated rapidhash secrets](#), which revealed that the quality of the 2-round scheme can fluctuate quite a bit when the multipliers don't mix well.

Rounds	Min	Mean	Max	Std Dev
2 rounds	2.03	7.92	40.37	7.19

We went with 2 rounds initially because that is the minimum to ensure every input bit reaches every output bit through at least one multiplication, providing nonlinear mixing rather than a single XOR fold. The fluctuations across the test runs, however, seemed to warrant another round, so we tested a 3-round version that looks like this:

```
const uint32_t kMask = (1 << 24) - 1; // 24-bit mask
const uint32_t kShift = 12; // half the bit width
// Multipliers in m[] are odd and randomly generated at startup.
uint32_t SeedArrayIndexValue(uint32_t value, uint32_t m[3]) {
    uint32_t m1 = m[0], m2 = m[1], m3 = m[2];
    uint32_t x = value;

    x ^= x >> kShift; x = (x * m1) & kMask; // round 1
    x ^= x >> kShift; x = (x * m2) & kMask; // round 2
    x ^= x >> kShift; x = (x * m3) & kMask; // round 3
    x ^= x >> kShift; // finalize
    return x;
}

// Modular inverses in m_inv[] are precomputed using Newton's method at startup.
uint32_t UnseedArrayIndexValue(uint32_t hash, uint32_t m_inv[3]) {
    uint32_t m1_inv = m_inv[0], m2_inv = m_inv[1], m3_inv = m_inv[2];
    uint32_t x = hash;

    x ^= x >> kShift; x = (x * m3_inv) & kMask; // undo round 3
    x ^= x >> kShift; x = (x * m2_inv) & kMask; // undo round 2
    x ^= x >> kShift; x = (x * m1_inv) & kMask; // undo round 1
    x ^= x >> kShift; // finalize
    return x;
}
```

This turned out to be quite effective in improving the stability of the avalanche effect:

Rounds	Min	Mean	Max	Std Dev		2 rounds	2.03	7.92	40.37	7.19		3 rounds
0.37	0.50	1.68	0.20									

Visualizing the hashes

Now let's look at some visualizations to get an intuitive sense of the hash. For the seeded hashes, we derived the constants from the default rapidhash secrets for the visualization, but the variation analysis above should also help you extrapolate the variance in these visualizations when different secrets are applied.

First, we take sequential inputs (spaced by an interval for better visibility) and plot their hash outputs. This helps us easily spot any linear relationships or patterns.

[image: Hash output vs sequential input]

As you can see, 2-round and 3-round xorshift-multiply both look significantly more random and less predictable than the naive constructions.

Another way to visualize it is to look at the [avalanche matrix](#). Here we take 50,000 random inputs, flip each input bit one at a time, and record how often each output bit changes. Each cell (row i , column j) shows the probability that flipping input bit i causes output bit j to flip - green means it's

close to the ideal 50%, red means it's strongly biased toward never or always flipping. The more green there is, the better.

[image: Avalanche matrix]

Once again, the 2-round and 3-round xorshift-multiply constructions show much better avalanche properties than the naive constructions.

Limitations

The SAC is a necessary but insufficient condition for a good hash function. Since this hashing scheme was developed to address a specific vulnerability, not to be a general-purpose PRNG or a non-cryptographic hash, we only measured bias from SAC as an empirical smoke test to guide the development, which happened in a limited timeframe. We have been [exploring other evaluations](#), but to keep this post focused we won't go into them here. To avoid falling into the trap of identifying weaknesses in a [spherical cow](#), it's important to keep in mind that structural weaknesses that cannot be exploited by a blind attacker to cause worst-case performance would only be informative rather than actionable in our threat model. The defense lies not only in the hash construction itself, but also in the lack of visibility of the randomly generated multipliers and the hash output.

Implementation

With a security release deadline to meet, we iterated on the hash design and V8 implementation in parallel, using the statistical analysis above to guide our choices while finding ways to reduce the impact on performance and code complexity.

Improving the access to hash secrets

Since our design will require even more frequent access to the hash secrets in hot paths, we need to ensure the access to them is efficient. In V8, the hash seed and the derived rapidhash secrets are stored in a `ByteArray` in the read-only roots, which in the default configuration of Node.js, are shared across isolates and initialized during process startup. The layout of the `ByteArray` was as follows:

Offset (bytes)	Content
0	seed (8 bytes)
8	secrets[0] (8 bytes)
16	secrets[1] (8 bytes)
24	secrets[2] (8 bytes)

The roots tend to be hot and in cache, so reading the secrets from the roots directly should be efficient. However, like most heap objects in V8, the `ByteArray` was previously allocated with the default 4-byte alignment. To deal with the potential misalignment when reading 8-byte secrets, the `HashSeed` struct that was used to map them for access in C++ was passed around by value, with

each part `memcpy`-ed from the `ByteArray`, or copied from another `HashSeed` that's not necessarily in cache. To reduce the overhead, we [updated the `ByteArray` to be allocated with 8-byte alignment, and changed the `HashSeed` to only hold a pointer to the beginning of the `ByteArray` in the read-only roots](#). Accesses to individual parts of the structure were then just direct loads from a pointer that points to the roots without copying, and in code they are just simple field accesses from a `HashSeed` struct reinterpreted over the `ByteArray`.

Extending the `HashSeed` for the new hash

For our new hashing scheme, during hash seed initialization, we derive the multipliers from the rapidhash secrets by taking the lowest 24 bits, compute their modular inverses using [Newton's method](#), and store them all in the `ByteArray` that `HashSeed` maps onto. When we initially implemented the 2-round xorshift-multiply scheme, the layout of the `ByteArray` became:

Offset (bytes)	Content
0	seed (8 bytes)
8	secrets[0] (8 bytes)
16	secrets[1] (8 bytes)
24	secrets[2] (8 bytes)
32	m1 (4 bytes)
36	m2 (4 bytes)
40	m1_inv (4 bytes)
44	m2_inv (4 bytes)

This was later extended to include `m3` and `m3_inv` for the 3-round scheme, with `m3` and `m3_inv` derived from `secrets[2]`.

Applying the new hashing scheme across V8

To implement the new seeded hashing scheme, we consolidated all the code paths that treated the value bits as the original numeric value to go through a few helpers and [enforce seeding when it's enabled](#):

- In runtime C++, e.g., slow paths for JS operations that involve conversions between strings and integers, numeric string parsing, equality checks in internal tables: when the code needs to encode or decode an array index hash, it will first load the `HashSeed` from the read-only roots of either the current [isolate group](#) or the shared [read-only heap](#), then call the new [encoding](#) and [decoding](#) helpers we added that perform the xorshift-multiply-based [seeding](#) and [unseeding](#).
- In JIT-compiled code, e.g., fast paths for JS operations that involve conversions between strings and integers like `parseInt`: the secrets need to be loaded from the roots of the current isolate. We added [Torque](#) macros to facilitate generating code that [encodes](#) and [decodes](#) array index hashes, while code generation for [seeding/unseeding](#) algorithms was implemented in the [Code StubAssembler](#) instead since the necessary operations were not yet well supported in Torque.

Deployment

The new seeded hashing scheme for array index strings has been merged into V8, gated by `v8_enable_seeded_array_index_hash = true`, and it needs to be used together with `v8_use_default_hasher_secret = false` for HashDoS resistance. For Chrome, where DoS attacks are not applicable, this will be disabled. In Node.js, this is enabled and shipped to v25, v24, v22, and v20 in the [March 2026 security release](#).

We have also notified other V8 embedders (Deno and Cloudflare workers) about the vulnerability and the fix during the development and the rollout.

Acknowledgments

This fix was developed and backported to Node.js LTS branches by [Joyee Cheung](#) (Igalia, under the sponsorship of Bloomberg). Thanks to [Mate Marjanović](#) for identifying and reporting the vulnerability, [Leszek Swirski](#) from the Google V8 team for reviewing and providing feedback on the design and implementation, [Chengzhong Wu](#) (Bloomberg) for reviewing the V8 patches for Node.js, [Matteo Collina](#) (Platformatic) for triaging and investigating the mitigations, [Olivier Flückiger](#) (Google V8) for helping with the coordination, [Antoine du Hamel](#), [Juan José Arboleda](#), [Marco Ippolito](#), and [Rafael Gonzaga](#) for preparing the security releases.

[Thursday, June 18, 2026 Security Releases](#)[Tuesday, March 24, 2026 Security Releases](#)

Archived from <https://nodejs.org/en/blog/vulnerability/march-2026-hashdos>. Rendered offline from the local Markdown copy.